Final Project Handout

Course Final Project: Concurrent Task Dispatcher in Rust

Big idea

In this project, you will build a **concurrent task dispatcher** in Rust. The system will simulate a stream of incoming work, place that work into one or more queues, and assign tasks to a bounded worker pool according to a scheduling policy.

This project is not only about “getting code to run.” It is about showing that you understand how systems are built from smaller parts: ownership, threads, channels, shared state, locks, worker pools, queues, fairness, throughput, and shutdown.

Your final submission must therefore include both:

- a working Rust program
- a clear explanation of your design decisions and trade-offs

---

Learning goals

By the end of this project, you should be able to:

1. Design a multi-threaded Rust system with clear responsibilities.
2. Use concurrency tools such as threads, channels, `Arc`, `Mutex`, and related synchronization ideas correctly.
3. Explain the difference between task arrival, queuing, dispatch, execution, and completion.
4. Compare scheduler behavior under different workloads.
5. Measure performance using simple system metrics.
6. Reason about failure cases such as starvation, poor fairness, lock contention, and shutdown bugs.

Project scenario

Imagine a small operating-system-style scheduler or service dispatcher.

Tasks arrive over time. Some tasks are mostly **CPU-bound** and some are mostly **I/O-bound**. Your system should not simply send work blindly to the next worker. Instead, it should use a queue-based design and make a scheduling decision.

At minimum, your program should:

- generate a large collection of tasks

- simulate arrival over time
- place tasks into a queue or queues
- use a bounded worker pool
- dispatch tasks according to a scheduling policy
- record useful statistics
- shut down cleanly

You are building a **simulation**, not a real operating system.

Required features

Your project must include all of the following.

1. Task model

Each task must include at least:

- `id`
- `arrival_time`
- `kind` where kind is either `CPU` or `IO`
- `duration`

You may include more fields if useful, such as:

- `priority`
- `cpu_cost`
- `deadline`
- `source`

2. Random workload generation

Your program must generate many tasks automatically.

Minimum expectation:

- at least **500 tasks** total
- task types should be mixed
- arrivals should not all happen at the exact same moment unless that is part of a specific experiment

Your program should use a fixed random seed for reproducible runs.

3. Bounded worker pool

You must use a fixed-size worker pool.

Minimum expectation:

- between **4 and 12 workers**
- workers should repeatedly accept tasks until shutdown

4. Queue-based architecture

You must place tasks into a queue before they are executed.

Acceptable designs include:

- one shared ready queue
- separate CPU and IO queues
- central dispatcher plus worker-specific queues

At least one queue is required. Two queues are encouraged.

5. Scheduling policy

You must implement at least one real policy for deciding what task goes next.

Examples:

- FIFO
- Round-robin dispatch
- shortest-queue dispatch
- reserve some workers for CPU tasks
- prefer short tasks first
- priority-based dispatch
- aging to reduce starvation

You must explain why you chose your policy.

6. Concurrent execution

Your design must actually use concurrency.

Examples of acceptable roles:

- task generator thread
- dispatcher thread
- worker threads
- logger or monitor thread

7. Instrumentation and metrics

Your program must collect and print summary statistics.

Required metrics:

- total tasks completed

- makespan
- average wait time
- average turnaround time

Choose at least **two more**:

- worker utilization
- queue length statistics
- number of CPU tasks completed
- number of IO tasks completed
- max wait time
- fairness gap between task classes
- idle time

8. Clean shutdown

Your system must terminate cleanly.

That means:

- no infinite loops after work is done
- no workers left waiting forever
- no hanging program at the end

Required experiments

You must run at least **two experiments** and compare the results.

Experiment A: balanced workload

A fairly mixed workload of CPU and IO tasks.

Experiment B: stressed workload

A workload designed to make your scheduler struggle.

Examples:

- many CPU-heavy tasks
- burst arrivals
- very uneven task durations
- one task type dominating the system

For each experiment, report:

- configuration used
- policy used

- summary metrics
- 1 short paragraph interpreting the result

Design constraints

To keep the project focused, follow these rules.

You must use

- Rust
- Cargo project structure
- standard library concurrency primitives where appropriate

You may use

- `rand`
- simple helper crates for CLI parsing or formatting
- standard collections such as `VecDeque`, `BinaryHeap`, `HashMap`

You should avoid

- huge external frameworks
- GUI work
- networking unless it is a small optional extension
- copying a full web server or full runtime from the internet

The center of the project is the scheduling system, not peripheral decoration.

Suggested architecture

You do not have to follow this exactly, but it is a strong starting point.

Option 1: central dispatcher

- one generator creates tasks
- generator sends tasks to dispatcher
- dispatcher stores tasks in one or two queues
- dispatcher assigns tasks to workers when workers become available
- workers simulate running the task
- workers report completion back to a collector or monitor

Option 2: load-balancer style

- one generator creates tasks
- a load balancer examines workload and task type
- load balancer pushes tasks toward worker queues

- each worker pulls from its own queue

Option 3: two-queue system

- CPU tasks go to one queue
- IO tasks go to another queue
- dispatcher decides which queue to pull from next
- optional reserved workers or weighted policy

What you must explain in your report

Your written report must answer these questions clearly.

1. What threads or major components exist in your design?
2. What data is shared, and how is it protected?
3. Where did you use channels, and why?
4. Where did you use shared state, and why?
5. What scheduling policy did you implement?
6. What behavior improved because of that policy?
7. What behavior became worse or more complicated because of that policy?
8. What concurrency bug or design mistake did you hit during development?
9. How did you fix it?
10. Where could starvation or unfairness still happen?

Deliverables

Submit all of the following.

1. Source code

A complete Cargo project.

Include:

- all `.rs` files
- `Cargo.toml`
- instructions for running

2. README

Your README must include:

- project title
- how to build and run
- command examples

- summary of design
- summary of experiments

3. Short design report

Length: about **2 to 4 pages**.

Include:

- architecture description
- data structures used
- synchronization strategy
- scheduling policy
- metrics collected
- experiment results
- lessons learned

4. Demo or defense

You must be ready to:

- run your code live
- explain a task's path through the system
- answer short design questions
- make or discuss a small modification request

Demo expectations

During the final check, you may be asked questions like these:

- Why did you choose channels here instead of `Arc<Mutex<_>>`?
- What exactly is inside the shared state?
- What would break if you removed this lock?
- Can a task starve in your design?
- What happens if all workers are busy and tasks keep arriving?
- What part of your system acts like a producer?
- What part acts like a consumer?
- What happens during shutdown?
- What would you change if CPU tasks became much more common?

You may also be asked to make a small change such as:

- reserve 2 workers for CPU tasks
- add simple priority
- add aging

- change FIFO to shortest-queue dispatch
- print one more metric

Grading rubric

1. Correctness and completion — 30 points

- program builds and runs
- required features are present
- clean shutdown works
- output is understandable

2. Concurrency design — 20 points

- threads/components have clear roles
- synchronization is appropriate
- design is not accidentally sequential
- queue/dispatch logic makes sense

3. Scheduling quality — 15 points

- policy is real and implemented coherently
- policy matches the project goal
- trade-offs are recognized

4. Metrics and experiments — 15 points

- metrics are computed correctly
- experiments are clearly described
- comparison is meaningful

5. Explanation and report — 10 points

- design is explained clearly
- terminology is used correctly
- reasoning is stronger than description

6. Demo / defense — 10 points

- student can explain their own code
- student can answer conceptual questions
- student can discuss bugs, trade-offs, and extensions

Total: **100 points**

AI and outside help policy

You may use outside resources, including AI tools, in the same way that programmers use references and assistants in real life. However, there is a strict condition:

You remain responsible for the work.

That means you must be able to:

- explain every major part of your design
- identify what synchronization tools are doing
- justify your scheduling choices
- debug or modify your code when asked

Required disclosure

In your README, include a short section titled **Tool Use Disclosure**. State:

- what tools you used
- what kind of help they provided
- one example of advice you accepted
- one example of advice you rejected or had to fix

Important note

A project that runs but cannot be explained well during demo or defense will lose points.

This course is evaluating your understanding, not just whether text appeared on the screen.

Milestones

These are suggested checkpoints.

Milestone 1: architecture sketch

Due first.

Submit:

- component diagram or bullet outline
- task struct idea
- queue design
- scheduling policy idea

Milestone 2: basic working pipeline

You should have:

- task generation
- queue insertion
- worker pool
- simple dispatch

Milestone 3: metrics and experiments

You should have:

- timing and statistics
- two workloads
- first comparison

Milestone 4: final polish

You should have:

- clean shutdown
- cleaned output
- README and report
- defense readiness

Suggested extensions for stronger projects

These are optional and can strengthen an already solid project.

- worker-specific queues with work stealing
- weighted scheduling between CPU and IO queues
- simple aging system to reduce starvation
- bounded queue capacity with backpressure
- dynamic task priority
- deadline-aware scheduling
- logging to file
- replay mode using recorded task input
- text-based visualization of queue lengths over time

Do not add extensions too early. A smaller correct design is better than a large broken one.

Common mistakes to avoid

- spawning unbounded numbers of threads
- using one giant lock around everything
- collecting no metrics until the end
- having no real shutdown path
- making the architecture so complicated that you cannot explain it
- turning the project into mostly formatting, printing, or CLI decoration
- treating CPU-bound and IO-bound tasks as labels only, without using the distinction in design or analysis

---

Final advice

Build this project in layers.

A good order is: 1. define the task struct 2. generate tasks 3. create queue logic 4. create worker pool 5. connect dispatcher to workers 6. add metrics 7. add experiment configurations 8. improve scheduling policy 9. polish shutdown and explanations

A system project becomes manageable when each part has one job.

Submission checklist

Before submitting, make sure you can say yes to all of these.

- My program builds with Cargo.
- My program actually uses concurrency.
- My program includes a queue-based design.
- My program uses a bounded worker pool.
- My program prints required metrics.
- My system shuts down cleanly.
- I ran at least two experiments.
- My README explains how to run the code.
- My report explains the design and trade-offs.
- I can explain and defend my implementation.

Starter question for yourself

If someone points at any one task in your system and asks,

“Where is this task right now, who owns it, and what happens to it next?”

you should be able to answer.

That is the heart of this project.